Lab 20 — Stateful Firewall with UFW and nftables

> **SY0-701 Objective 3.2** — Given a scenario, apply security principles to secure enterprise infrastructure
>
> **Domain 3** — Security Architecture (18%) · **Time:** ~25 min · **Course:** C628 CompTIA Security+ Exam Prep

Goal: Build a default-deny allow-list with UFW, see the netfilter rules it generates, then write a stateful nftables ruleset for a server, prove which ports it allows and blocks, and see what a layer 4 firewall cannot see.

You'll build: A UFW host policy and an nftables-protected server (in its own network namespace) with per-rule counters.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 19 is helpful background; tools installed in step 1.

Background

- **UFW** and **nftables** are two front ends to the same kernel **netfilter** engine. UFW is a friendly allow-list tool; nftables is the full rule language.
- A **stateful (layer 4)** firewall tracks connections: once a NEW connection is allowed, ESTABLISHED and RELATED reply packets pass automatically, so you never write reverse rules.
- The secure baseline is **default deny inbound**, allow only named services, and accept established traffic. Rules are evaluated top down and the first match wins; anything unmatched hits the **implicit deny**.
- A layer 4 firewall decides on IP addresses, protocols and ports only. It cannot read HTTP content — that is the job of a **layer 7** device such as a WAF or NGFW (Lab 21).
- **Failure mode** matters: a host firewall that fails closed protects the data but can cut off your own SSH session, so management access is always allowed first.

Steps

1. Install the tools

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y ufw nftables
netcat-openbsd curl python3 iproute2 iputils-ping
mkdir -p /root/lab20
```

2. Build a UFW default-deny allow-list

Set the defaults, allow SSH first so you never lock yourself out, then allow the published web ports.

```
ufw default deny incoming
ufw default allow outgoing
ufw allow 22/tcp comment 'SSH admin — always first'
ufw allow 80/tcp comment 'HTTP'
```

```
ufw allow 443/tcp comment 'HTTPS'
ufw --force enable
ufw status verbose
```

3. Inspect the netfilter rules UFW generated

UFW only writes netfilter rules for you. Look at the user chain it built and the default policy.

```
ufw status numbered
iptables -S ufw-user-input
iptables -S INPUT | head -3
```

```
-A ufw-user-input -p tcp -m tcp --dport 22 -j ACCEPT
-A ufw-user-input -p tcp -m tcp --dport 80 -j ACCEPT
-A ufw-user-input -p tcp -m tcp --dport 443 -j ACCEPT
-P INPUT DROP
```

4. Disable UFW and build a server namespace

To test from the outside without risking the playground's own connection, disable UFW and create a separate "server" network stack reachable at 10.20.0.2. It runs a web server on 80, a fake SSH service on 22 and a legacy Telnet service on 23.

```
ufw --force disable
ip netns add srv
ip link add veth-host type veth peer name veth-srv
ip link set veth-srv netns srv
ip addr add 10.20.0.1/24 dev veth-host && ip link set veth-host up
ip netns exec srv ip addr add 10.20.0.2/24 dev veth-srv
ip netns exec srv ip link set veth-srv up
ip netns exec srv ip link set lo up
ip netns exec srv python3 -c 'import http.server as h, socketserver as s;
s.TCPServer(("10.20.0.2", 80), h.SimpleHTTPRequestHandler).serve_forever()' >/dev/null
2>&1 &
ip netns exec srv nc -lk 22 >/dev/null 2>&1 &
ip netns exec srv nc -lk 23 >/dev/null 2>&1 &
sleep 1
for p in 22 23 80; do nc -z -w2 10.20.0.2 $p && echo "port $p open (no firewall yet)";
done
```

5. Write a stateful nftables ruleset for the server

Default-drop input, accept loopback and established traffic, allow SSH only from the admin host, allow web from anyone, rate-limit ping, and count everything else as dropped.

```
cat > /root/lab20/srv.nft <<'EOF'
flush ruleset
table inet filter {
  chain input {
    type filter hook input priority 0; policy drop;
    iif "lo" accept
    ct state established,related accept
    ct state invalid counter drop comment "invalid packets"
```

```
    ip saddr 10.20.0.1 tcp dport 22 counter accept comment "SSH from admin host only"
    tcp dport { 80, 443 } counter accept comment "published web"
    icmp type echo-request limit rate 5/second counter accept comment "rate-limited ping"
    counter drop comment "default deny"
  }
  chain forward {
    type filter hook forward priority 0; policy drop;
  }
  chain output {
    type filter hook output priority 0; policy accept;
  }
}
EOF
nft -c -f /root/lab20/srv.nft && echo "Syntax OK"
```

6. Load the ruleset and test allowed versus blocked ports

Load the rules inside the server namespace only, then probe it from the host.

```
ip netns exec srv nft -f /root/lab20/srv.nft
curl -s -o /dev/null -w "HTTP 80  -> %{http_code}\n" http://10.20.0.2/
nc -z -w2 10.20.0.2 22 && echo "SSH 22   -> allowed (admin host)"
nc -z -w2 10.20.0.2 23 || echo "Telnet 23 -> blocked (default deny)"
ping -c2 -W1 10.20.0.2 >/dev/null && echo "ICMP     -> allowed (rate limited)"
```

```
HTTP 80  -> 200
SSH 22   -> allowed (admin host)
Telnet 23 -> blocked (default deny)
ICMP     -> allowed (rate limited)
```

7. Read per-rule counters

Counters show packets and bytes per rule — the raw evidence for firewall log review.

```
for i in $(seq 1 20); do curl -s -o /dev/null http://10.20.0.2/; done
for i in 1 2 3; do nc -z -w1 10.20.0.2 23; done
ip netns exec srv nft list chain inet filter input
```

The `published web` counter grows by the requests you sent, and the `default deny` counter shows the blocked Telnet attempts.

8. Compare layer 4 rules with layer 7 filtering

A layer 4 firewall allows port 80, so it lets an SQL injection payload straight through to the application.

```
curl -s -o /dev/null -w "SQLi payload through the L4 firewall -> HTTP %{http_code}\n" \
  "http://10.20.0.2/?id=1%27%20OR%20%271%27=%271"
ip netns exec srv nft list chain inet filter input | grep 'published web'
```

The request is accepted and counted as ordinary web traffic. Stopping it needs a layer 7 control — the WAF in Lab 21.

Test it

```
curl -s -o /dev/null -w "%{http_code}\n" http://10.20.0.2/
nc -z -w2 10.20.0.2 23 && echo "FAIL: telnet reachable" || echo "PASS: telnet blocked"
ip netns exec srv nft list chain inet filter input | grep -E 'default deny'
```

Expected: `200`, `PASS: telnet blocked`, and a `default deny` rule whose packet counter is greater than 0.

Troubleshooting

Symptom	Fix
The terminal freezes after `ufw --force enable`	Refresh the Killercoda tab; if it stays frozen run `ufw --force disable` in a new terminal.
`Error: Could not process rule: No such file or directory`	Load the nftables kernel module with `modprobe nf_tables`, then retry step 6.
`curl` to 10.20.0.2 returns `000`	The web server did not start; rerun the `python3 -c ...` `TCPServer` line from step 4.

Challenge

Add an egress (output) policy to the server namespace that drops all outbound traffic except DNS (UDP 53) and replies to established connections, then prove the server can no longer start an outbound connection to 10.20.0.1 on port 4444 — a classic reverse-shell port.

Reflection

Your web server sits behind a correctly configured stateful firewall yet is breached by SQL injection. Which firewall type from Objective 3.2 would have helped, where would you place it, and should it fail open or fail closed?

Exam tips

- Stateful firewalls track connections; stateless packet filters judge each packet alone and need explicit return rules.
- Order matters: first match wins, so specific denies must sit above broad allows. The last rule is an implicit deny.
- Layer 4 = IPs and ports; layer 7 = application content (WAF, NGFW). Exam scenarios about payloads need a layer 7 answer.
- UTM bundles firewall, IPS, anti-malware and VPN in one box; an NGFW adds application and user awareness.

References

- nftables wiki — https://wiki.nftables.org/
- Ubuntu UFW documentation — https://help.ubuntu.com/community/UFW
- NIST SP 800-41 Rev. 1 Guidelines on Firewalls and Firewall Policy — https://csrc.nist.gov/pubs/sp/800/41/r1/final

- CIS Ubuntu Linux Benchmark (host firewall section) — https://www.cisecurity.org/benchmark/ubuntu_linux

---